

Guided Practice

Lesson 2: Designing and Deploying Intelligent Agents

Overview

In this guided practice, you will build a multi-agent system using LangChain and Azure OpenAI to simulate feature prioritization. You will use three domain-specific agents, User Advocate, Tech Lead, and Business Analyst, to score and explain feature values from different perspectives. The LangChain agent will query each tool, aggregate insights, and return a prioritized list. By completing this guided practice, you will learn how to orchestrate tool-based agents for better decision-making.

Instructions

1. Use Visual Studio Code and Python to complete each step
2. Work individually or in pairs to experiment as needed
3. Spend approximately 20-30 minutes completing all steps
4. Document your code, outputs, and final results for review

Tasks

Scenario: You are a product manager evaluating three upcoming product features: Dark Mode, AI-based Smart Search, and Offline Support. To make a data-backed decision, you must use LangChain agents that simulate input from internal stakeholders. Each agent scores the features based on their specific lens, usability, feasibility, or business value. You will then run a master agent that uses these tools to return a prioritized list based on cumulative scoring.

Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Define scoring functions and convert them into tools
3. Connect to Azure OpenAI and initialize the agent
4. Prompt the agent and analyze the result

Tools required: Python, Visual Studio Code, LangChain, and Azure OpenAI

Discussion questions (optional):

If time permits, discuss the following questions:

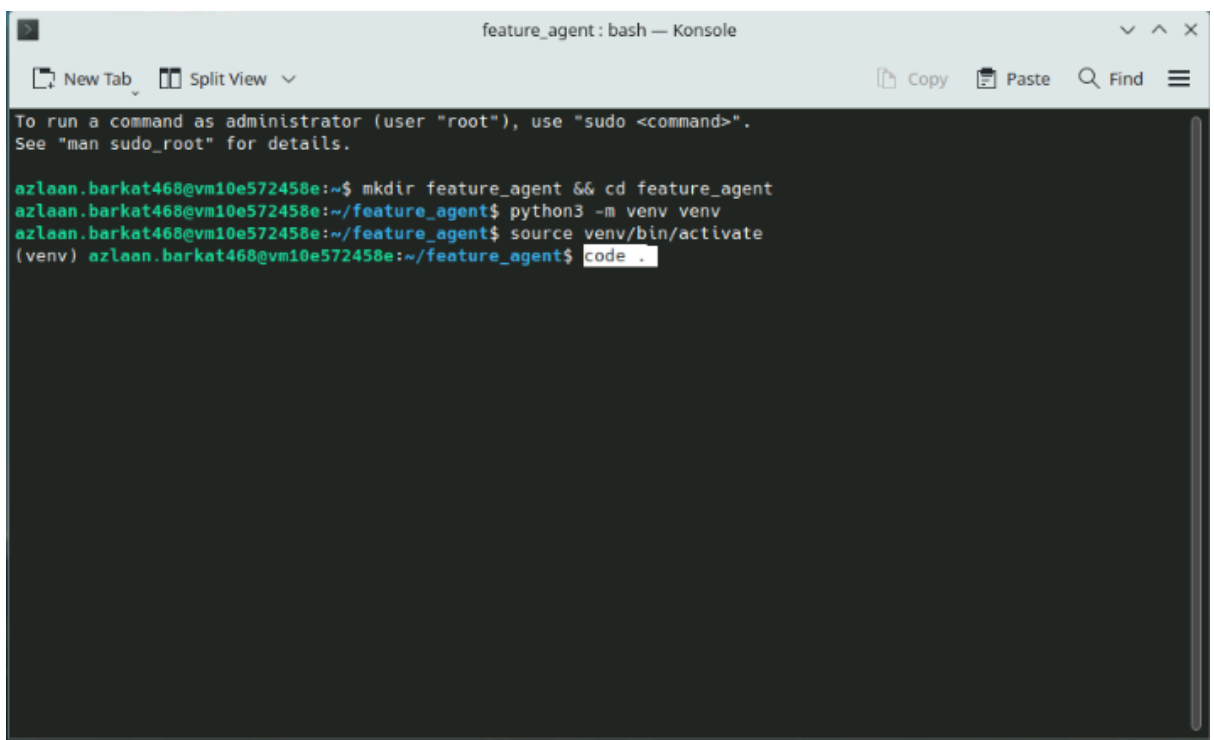
1. If each team in a company had its own AI agent, would decision-making become more efficient or more chaotic? Why?
2. Would you trust an AI-generated feature roadmap? What evidence would you need to feel confident in its recommendations?

Answer Key

Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal on your system and run the following command:

```
mkdir feature_agent && cd feature_agent
python3 -m venv venv
source venv/bin/activate
code .
```

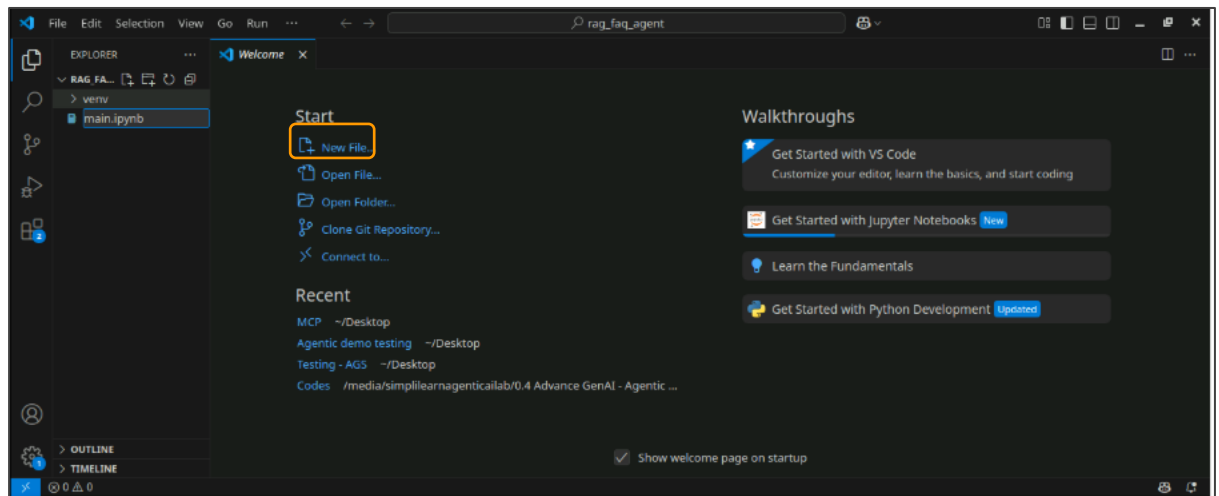
A screenshot of a terminal window titled "feature_agent: bash — Konsole". The window has a light blue header bar with icons for "New Tab", "Split View", "Copy", "Paste", "Find", and a menu icon. The terminal content shows a series of commands and their outputs. It starts with a message about running as administrator, followed by the commands: "mkdir feature_agent && cd feature_agent", "python3 -m venv venv", "source venv/bin/activate", and "code .". The prompt changes from "~\$" to "~/\$" and then to "(venv) azlaan.barkat468@vm10e572458e:~/feature_agent\$".

```
feature_agent: bash — Konsole
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azlaan.barkat468@vm10e572458e:~$ mkdir feature_agent && cd feature_agent
azlaan.barkat468@vm10e572458e:~/feature_agent$ python3 -m venv venv
azlaan.barkat468@vm10e572458e:~/feature_agent$ source venv/bin/activate
(venv) azlaan.barkat468@vm10e572458e:~/feature_agent$ code .
```

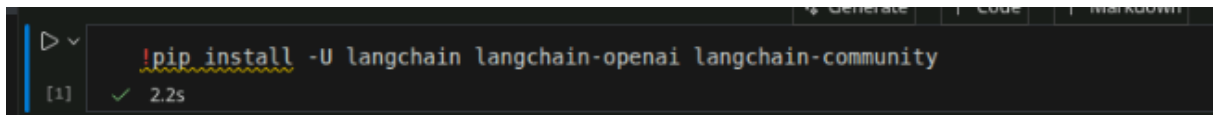
Note: This creates a virtual environment inside the **feature_agent** folder and opens the project in VS Code.

1.2 Click **New File...** to create a new file, and name it **main.ipynb**



1.3 In the first cell, run the following command to install required packages:

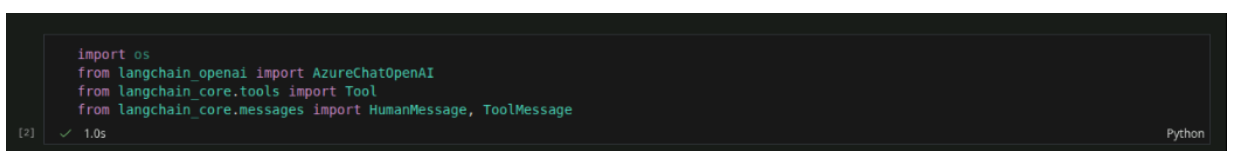
!pip install -U langchain langchain-openai langchain-community



Step 2: Define scoring functions and convert them into tools

2.1 Execute the following code to import dependencies:

```
import os
from langchain_openai import AzureChatOpenAI
from langchain_core.tools import Tool
from langchain_core.messages import HumanMessage, ToolMessage
```



2.2 Use functions to define role-specific scoring logic using the following code:

```
def user_advocate_tool(feature):
    scores = {
        "Dark Mode": (5, "Some users want it for comfort"),
        "AI-based Smart Search": (9, "Highly requested feature"),
        "Offline Support": (7, "Useful for travel and remote work")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → User Score: {score}. Note: {comment}"

def tech_lead_tool(feature):
    scores = {
        "Dark Mode": (8, "Easy to implement"),
        "AI-based Smart Search": (5, "Requires custom model integration"),
        "Offline Support": (6, "Moderate complexity")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → Tech Score: {score}. Note: {comment}"

def business_analyst_tool(feature):
    scores = {
        "Dark Mode": (4, "Minimal impact on revenue"),
        "AI-based Smart Search": (9, "Improves retention and engagement"),
        "Offline Support": (6, "Moderate business value")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → Business Score: {score}. Note: {comment}"

tools = [
    Tool(name="UserAdvocate", func=user_advocate_tool, description="User desirability scoring"),
    Tool(name="TechLead", func=tech_lead_tool, description="Technical feasibility scoring"),
    Tool(name="BusinessAnalyst", func=business_analyst_tool, description="Business value scoring")
]
```

```
def user_advocate_tool(feature):
    scores = {
        "Dark Mode": (5, "Some users want it for comfort"),
        "AI-based Smart Search": (9, "Highly requested feature"),
        "Offline Support": (7, "Useful for travel and remote work")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → User Score: {score}. Note: {comment}"

def tech_lead_tool(feature):
    scores = {
        "Dark Mode": (8, "Easy to implement"),
        "AI-based Smart Search": (5, "Requires custom model integration"),
        "Offline Support": (6, "Moderate complexity")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → Tech Score: {score}. Note: {comment}"
```

```
def business_analyst_tool(feature):
    scores = {
        "Dark Mode": (4, "Minimal impact on revenue"),
        "AI-based Smart Search": (9, "Improves retention and engagement"),
        "Offline Support": (6, "Moderate business value")
    }
    score, comment = scores.get(feature, (0, "No data"))
    return f"{feature} → Business Score: {score}. Note: {comment}"

tools = [
    Tool(name="UserAdvocate", func=user_advocate_tool, description="User desirability scoring"),
    Tool(name="TechLead", func=tech_lead_tool, description="Technical feasibility scoring"),
    Tool(name="BusinessAnalyst", func=business_analyst_tool, description="Business value scoring")
]
```

[3] ✓ 0.0s Python

Note: Each function simulates stakeholder input and is wrapped into a LangChain-compatible tool using the Tool class.

Step 3: Connect to Azure OpenAI and initialize the agent

3.1 Configure the model and initialize the agent using the following code:

```
llm = AzureChatOpenAI(
    api_key="2ABecfnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99
    BDAC77bzfXJ3w3AAABACOGi3sC",
    azure_endpoint="https://openai-api-management-gw.azure-api.net",
    deployment_name="gpt-5-mini",
    api_version="2025-01-01-preview"
)
```

3.2 Bind the tools to the llm model using the following code:

```
llm_with_tools = llm.bind_tools(tools)
```

```
llm_with_tools = llm.bind_tools(tools)
```

[6] ✓ 0.0s

3.3 Initialize the agent using the following code:

```
def run_agent(prompt: str):
    messages = [HumanMessage(content=prompt)]

    while True:
        response = llm_with_tools.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return response.content

        for tool_call in response.tool_calls:
            tool_name = tool_call["name"]
            tool_args = tool_call.get("args", {})

            selected_tool = next(t for t in tools if t.name == tool_name)

            if "__arg1" in tool_args:
                tool_output = selected_tool.func(tool_args["__arg1"])
            else:
                tool_output = selected_tool.func(**tool_args)

            messages.append(
                ToolMessage(
                    content=str(tool_output),
                    tool_call_id=tool_call["id"],
                )
            )
```

```
def run_agent(prompt: str):
    messages = [HumanMessage(content=prompt)]

    while True:
        response = llm.with_tools.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return response.content

        for tool_call in response.tool_calls:
            tool_name = tool_call["name"]
            tool_args = tool_call.get("args", {})

            selected_tool = next(t for t in tools if t.name == tool_name)

            if "_arg1" in tool_args:
                tool_output = selected_tool.func(tool_args["_arg1"])
            else:
                tool_output = selected_tool.func(**tool_args)

            messages.append(
                ToolMessage(
                    content=str(tool_output),
                    tool_call_id=tool_call["id"],
                )
            )

[7] ✓ 0.0s Python
```

Step 4: Prompt the agent and analyze the result

4.1 Provide the agent with the following prompt:

```
prompt = """
Evaluate each of the following features one by one:
Dark Mode, AI-based Smart Search, and Offline Support.
```

Use the available tools to get:

- User Score (from UserAdvocate)
- Technical Feasibility Score (from TechLead)
- Business Value Score (from BusinessAnalyst)

Then combine the scores and return a prioritized list from highest to lowest.

```
"""
```

```
prompt = """
Evaluate each of the following features one by one:
Dark Mode, AI-based Smart Search, and Offline Support.

Use the available tools to get:
- User Score (from UserAdvocate)
- Technical Feasibility Score (from TechLead)
- Business Value Score (from BusinessAnalyst)

Then combine the scores and return a prioritized list from highest to lowest.
"""

[8] ✓ 0.0s Python
```

4.2 Run the agent to view the results:

```
result = run_agent(prompt)
print(result)
```

```
▶ ✓  
result = run_agent(prompt)  
print(result)  
[9] ✓ 20.4s
```

Note: This code block instructs the agent to query all tools for each feature and aggregate the results into a ranked list.

Expected output may look as shown in the following screenshot:

```
... Method: I used the three tool-provided scores (User, Technical Feasibility, Business) for each feature, averaged them (equal weighting) to make  
  
1) Dark Mode  
- User Score: 5 - Some users want it for comfort.  
- Technical Feasibility Score: 8 - Easy to implement.  
- Business Value Score: 4 - Minimal impact on revenue.  
- Combined (average):  $(5 + 8 + 4) / 3 = 5.7$   
- Quick rationale: Low business impact but very low technical risk and modest user demand. Good candidate for a low-effort visual polish.  
- Suggested next step: Implement as a small design/engineering task (theme variables + toggle + user preference persistence).  
  
2) AI-based Smart Search  
- User Score: 9 - Highly requested feature.  
- Technical Feasibility Score: 5 - Requires custom model integration.  
- Business Value Score: 9 - Improves retention and engagement.  
- Combined (average):  $(9 + 5 + 9) / 3 = 7.7$   
- Quick rationale: High user demand and high business upside, but moderate technical complexity (data, model selection, infra, latency, privacy)  
- Suggested next step: Prototype PoC using an existing model or search-enhancement API, evaluate relevance/latency/cost before full rollout.  
  
3) Offline Support  
- User Score: 7 - Useful for travel and remote work.  
- Technical Feasibility Score: 6 - Moderate complexity.  
- Business Value Score: 6 - Moderate business value.  
- Combined (average):  $(7 + 6 + 6) / 3 = 6.3$   
- Quick rationale: Meaningful user benefit and moderate business impact; requires work on caching, sync, conflict resolution, and testing across  
- Suggested next step: Build an offline-first prototype for the most common workflows and evaluate sync/reconciliation complexity.  
...  
  
If you want, I can:  
- Recompute priority with different weights (e.g., business > user > tech).  
- Produce a rough effort estimate and timeline for each feature.  
  
Spaces: 4 {} 📄 Cell 9 of 10
```